Question

Write a C program to **segregate even and odd nodes in a singly linked list** such that all even-valued nodes appear before odd-valued nodes. The relative order of nodes within even and odd groups should be maintained.

Approach (Logic)

- Traverse the linked list.
- Create two separate lists:
  - One for **even nodes**
  - One for **odd nodes**
- Finally, connect the even list to the odd list.

C Program

```
#include &lt;stdio.h&gt;
#include &lt;stdlib.h&gt;

// Node structure
struct Node {
    int data;
    struct Node* next;
};

// Function to create a new node
struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode-&gt;data = data;
    newNode-&gt;next = NULL;
    return newNode;
}

// Insert node at end
void insertEnd(struct Node** head, int data) {
```

```
    struct Node* newNode = createNode(data);
    if (*head == NULL) {
        *head = newNode;
        return;
    }
    struct Node* temp = *head;
    while (temp-&gt;next != NULL)
        temp = temp-&gt;next;
    temp-&gt;next = newNode;
}

// Function to segregate even and odd nodes
void segregateEvenOdd(struct Node** head) {
    struct Node *evenStart = NULL, *evenEnd = NULL;
    struct Node *oddStart = NULL, *oddEnd = NULL;
    struct Node* curr = *head;

    while (curr != NULL) {
        int val = curr-&gt;data;

// Even node
        if (val % 2 == 0) {
            if (evenStart == NULL) {
                evenStart = curr;
                evenEnd = evenStart;
            } else {
                evenEnd-&gt;next = curr;
                evenEnd = evenEnd-&gt;next;
            }
        }
// Odd node
        else {
            if (oddStart == NULL) {
                oddStart = curr;
                oddEnd = oddStart;
            } else {
                oddEnd-&gt;next = curr;
                oddEnd = oddEnd-&gt;next;
            }
```

```
        }
        curr = curr-&gt;next;
    }

// If no even or no odd nodes
    if (evenStart == NULL || oddStart == NULL)
        return;

// Combine even and odd lists
    evenEnd-&gt;next = oddStart;
    oddEnd-&gt;next = NULL;

    *head = evenStart;
}

// Display linked list
void display(struct Node* head) {
    while (head != NULL) {
        printf("%d -&gt; ", head-&gt;data);
        head = head-&gt;next;
    }
    printf("NULL\n");
}

// Main function
int main() {
    struct Node* head = NULL;

    insertEnd(&amp;head, 17);
    insertEnd(&amp;head, 15);
    insertEnd(&amp;head, 8);
    insertEnd(&amp;head, 12);
    insertEnd(&amp;head, 10);
    insertEnd(&amp;head, 5);
    insertEnd(&amp;head, 4);

    printf("Original List:\n");
    display(head);
```

```
    segregateEvenOdd(&amp;head);

    printf("After Segregating Even and Odd:\n");
    display(head);

    return 0;
}
```

Sample Output

Original List:
`17 -> 15 -> 8 -> 12 -> 10 -> 5 -> 4 -> NULL`

After Segregating Even and Odd:
`8 -> 12 -> 10 -> 4 -> 17 -> 15 -> 5 -> NULL`

Question

Write a C program to **rotate a singly linked list to the right by k places.**

Explanation (Logic)

- Find the **length (n)** of the linked list.
- Compute effective rotations:
  `k = k % n`
- Find the **(n - k)th node** → this will be the new last node.
- Make the list **circular**, then break it at the correct position.

C Program

```
#include &lt;stdio.h&gt;
#include &lt;stdlib.h&gt;
```

```
// Node structure
struct Node {
    int data;
    struct Node* next;
};

// Create new node
struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode-&gt;data = data;
    newNode-&gt;next = NULL;
    return newNode;
}

// Insert at end
void insertEnd(struct Node** head, int data) {
    struct Node* newNode = createNode(data);
    if (*head == NULL) {
        *head = newNode;
        return;
    }
    struct Node* temp = *head;
    while (temp-&gt;next != NULL)
        temp = temp-&gt;next;
    temp-&gt;next = newNode;
}

// Rotate list to the right by k places
void rotateRight(struct Node** head, int k) {
    if (*head == NULL || (*head)-&gt;next == NULL || k == 0)
        return;

    struct Node* temp = *head;
    int length = 1;

// Find length
    while (temp-&gt;next != NULL) {
        temp = temp-&gt;next;
```

```
        length++;
    }

// Make it circular
    temp-&gt;next = *head;

// Effective rotations
    k = k % length;

    int stepsToNewHead = length - k;
    struct Node* newTail = *head;

// Move to new tail
    for (int i = 1; i &lt; stepsToNewHead; i++) {
        newTail = newTail-&gt;next;
    }

// Set new head
    struct Node* newHead = newTail-&gt;next;

// Break the loop
    newTail-&gt;next = NULL;

    *head = newHead;
}

// Display list
void display(struct Node* head) {
    while (head != NULL) {
        printf("%d -&gt; ", head-&gt;data);
        head = head-&gt;next;
    }
    printf("NULL\n");
}

// Main function
int main() {
    struct Node* head = NULL;

    insertEnd(&amp;head, 1);
```

```
    insertEnd(&amp;head, 2);
    insertEnd(&amp;head, 3);
    insertEnd(&amp;head, 4);
    insertEnd(&amp;head, 5);

    int k = 2;

    printf("Original List:\n");
    display(head);

    rotateRight(&amp;head, k);

    printf("After rotating right by %d places:\n", k);
    display(head);

    return 0;
}
```

Sample Output

Original List:
```
1 -&gt; 2 -&gt; 3 -&gt; 4 -&gt; 5 -&gt; NULL
```

After rotating right by 2 places:
```
4 -&gt; 5 -&gt; 1 -&gt; 2 -&gt; 3 -&gt; NULL
```